Mini Project 2: Pennine Pizza Works

GitHub Repository: COM4401-2026-AS36-2/02_Software_Engineering/02_Contributions at main · Mitchellll007-droid/COM4401-2026-AS36-2

Table of Contents

1: Analysis 1

1.1: Requirements Table 1

1.2: IPO Table 1

1.3: Data Sheet, Business rules supplied by the client 2

2: Design 2

2.1: Structure Diagram 3

2.2: Data Structures 3

2.3: Function/subroutine list 5

2.4: Algorithm/Pseudocode 7

3: Build/Implementation 12

3.1: Program Flowcharts 12

3.1.1: Main 12

3.1.2: GetNumPizzas 12

3.1.3: getPizzaChoices 12

3.1.4: getDetails 12

3.1.5: CheckBlank 12

3.1.5: calcSubtotal 12

3.1.7 GetDelivery 12

3.1.8 calcDiscount 12

3.1.9 printBill 12

3.2: Sample Run 12

4: Testing 12

4.1: Test plan and results 12

1: Analysis

1.1: Requirements Table

Requirements	
R1	Customer name, address and phone number
R2	The number of pizzas ordered
R3	The size of each pizza
R4	The number of extra toppings for each pizza
R5	Whether delivery is required

1.2: IPO Table

Input	Process	Output
Customer name	Determine the base price of each pizza based on its size	Display the customer's order details.
Address	Determine the extra topping cost for each pizza based on the number of toppings	Display the number and sizes of the pizzas ordered.
Phone number	Calculate each pizza cost = base pizza price + extra topping cost	Display the number of extra toppings selected.
No of pizzas	Calculate order subtotal = total cost of all pizzas	Display whether delivery is required.
Size of pizzas	Check if order subtotal is over £20	Display the cost of the pizzas.

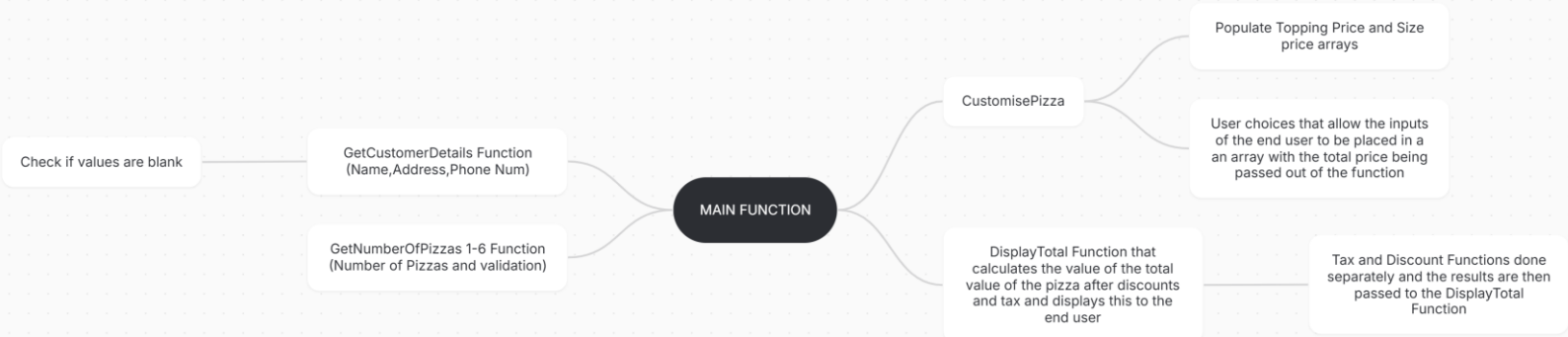
Toppings	If subtotal is over £20, calculate discount = subtotal × 10%	Display the cost of any extra toppings.
Delivery (Boolean)	Calculate final order total = subtotal after discount + delivery charge	Display the delivery charge, if applicable.

1.3: Data Sheet, Business rules supplied by the client

Item	Rule
Minimum number of pizzas	1
Maximum number of pizzas	6
Small pizza	£3.25
Medium pizza	£5.50
Large pizza	£7.15
1 extra topping	£0.75
2 extra toppings	£1.35
3 extra toppings	£2.00
4 or more extra toppings	£2.50
Discount	10% discount if the order subtotal is over £20 before delivery is applied
Delivery charge	£2.50 if delivery is requested

2: Design

2.1: Structure Diagram



2.2: Data Structures

Arrays

Array	Type	Size	Declared in	Holds
customerDetails	string array	3	main	The customer details such as name, phone number and address
sizes	String array	6	main	This holds all the sizes of pizzas chosen across a customer's order up to a maximum of 6
toppings	String array	6	main	This holds all the toppings of pizzas chosen across a customer's order up to a maximum of 6
info	String array	3	getDetails	This holds all the customer details gotten in the getDetails function then passes it over to the customerDetails array in main

Dictionaries

Dictionary	Size	Holds
SIZE_PRICES	3	This holds the key-pair values of all pizza sizes and their corresponding prices. Eg 'S' or small is 3.25 etc
TOP_PRICES	5	This holds all the key pair values of all possible number of extra toppings the customer can add and the corresponding prices. Eg '1' extra topping is 0.75

Variables

Variable	Type	Used in	Purpose
name	string	Get_details() subroutine uses this variable	It is used to store the user input labelled name
address	string	Get_details() subroutine uses this variable	It is used to store the user input labelled address
phone	string	Get_details() subroutine uses this variable	It is used to store the user input labelled phone
text	string	This is used in the check_blank subroutine	It is used to store the passed argument which is stripped(spaces taken out) and compared with a blank space to validate blank spaces
num	integer	This is used in the get_pizza subroutine	It is used to store the number of pizzas being ordered and then used in a loop to initialise an empty 2d array
size	string	This is used in the get_pizza_choices subroutine	It is used to store the user inputted value of Size of pizza being ordered
wants_topping	string	This is used in the get_pizza_choices subroutine	It is used to store the user choice of if they want toppings as a single character
toppings	string	This is used in the get_pizza_choices subroutine	It is used to store the exact number of toppings chosen by the customer.
subtotal	Float/real	This is used in the calc_subtotal subroutine	It is used to store the running subtotal of all pizzas ordered as

			well as any extra toppings added
choice	string	This is used in the get_delivery subroutine	This is used to hold onto the data inputted by user in terms of choice of whether delivery is needed or not

2.3: Function/subroutine list

Function/subroutine name	Purpose	Order in main program
calcDiscount	This function takes in the base subtotal of the order after all choices have been added and their prices all totalled and checks if its greater than 20, if it is the discountedSubtotal is 90 percent of the original subtotal otherwise the subtotal stays the same. Regardless, the subtotal is returned.	6
calcSubtotal	This function takes in the arrays containing all the sizes and toppings choices of the order and compares them with a dictionary to assign a price value to each and add it all to a subtotal which is returned at the end.	5
checkBlank	This function is used to validate the customer details that the user inputs and assigns a false value if empty or a true value if there not empty and returns this value. This is then used in another function to repeat asking	2

	for customer details till a valid input is recorded.	
getDelivery	This function simply asks the user if they would like delivery for a fee of 2.50. it validates if the user enters either a capital or little Y or N and once this has been recorded assigns a deliveryfee value of 0.00 or 2.50 based on the input. It then returns the deliveryFee value.	7
getDetails	This function takes in customer details as inputs from the user and stores it all in the info array which is passed to another array in main. It also utilizes the checkBlank function to validate if the user entered something as an input.	1
getNumPizzas	This function gets the number of pizzas to be ordered which is useful for setting up the sizes and toppings arrays. It also validates if the number of pizzas is between a minimum of 1 and a maximum of 6. it then returns the number that is inputted.	3
getPizzaChoices	This function is used to append the arrays sizes and toppings with the user choices, so either S, M or L pizza size and if they want toppings, either 1, 2, 3 or 4+. This is appended up till the number of pizzas the user chooses to order.	4
printBill	This function takes all the inputs created in the rest of the program and creates a clear, itemised bill showing the customer's	8

	name, phone number and address. Then showing all pizzas orders with the size choice and the toppings choice. This is all added together then a deliveryFee is added if required. Then finally if the subtotal meets the threshold for a discount, it is applied, Showing the final subtotal.	

2.4: Algorithm/Pseudocode

```
Function Main
    Declare String Array customerDetails[3]
    Declare Integer num
    Declare Real rawSubtotal
    Declare Real discountedSubtotal
    Declare Real deliveryFee
    Declare Real finalTotal
    Declare String Array sizes[6]
    Declare String Array toppings[6]

    Call getDetails(customerDetails)
    Assign num = getNumPizzas()
    Call getPizzaChoices(sizes, toppings, num)
    Assign rawSubtotal = calcSubtotal(sizes, toppings, num)
    Assign discountedSubtotal = calcDiscount(rawSubtotal)
    Assign deliveryFee = getDelivery()
    Assign finalTotal = discountedSubtotal + deliveryFee
    Call printBill(customerdetails, sizes, toppings, num, rawSubtotal, discountedSubtotal, deliveryfee, finalTotal)
End
```

```
Function calcDiscount (Real rawSubtotal)
    Declare Real discountedSubtotal

    If rawSubtotal > 20.00
        Assign discountedSubtotal = rawSubtotal * 0.90
    Else
        Assign discountedSubtotal = rawSubtotal
    End
    Return Real discountedSubtotal
End
```

```
Function calcSubtotal (String Array sizes, String Array toppings, Integer num)
    Declare Integer i
    Declare Real pizzaPrice
    Declare Real toppingPrice
    Declare Integer subtotal

    Assign toppings[0] = ""
    Assign toppings[1] = ""
    Assign toppings[2] = ""
    Assign toppings[3] = ""
    Assign toppings[4] = ""
    Assign toppings[5] = ""
    Assign subtotal = 0.0
    For i = 0 to num-1
        If sizes[i] == "S" or sizes[i] == "s"
            Assign pizzaPrice = 3.25
        Else
            If sizes[i] == "M" or sizes[i] == "m"
                Assign pizzaPrice = 5.50
            Else
                Assign pizzaPrice = 7.15
            End
        End
        If toppings[i] == "1"
            Assign toppingPrice = 0.75
        Else
            If toppings[i] == "2"
                Assign toppingPrice = 1.35
            Else
                If toppings[i] == "3"
                    Assign toppingPrice = 2.00
                Else
                    If toppings[i] == "4+"
                        Assign toppingPrice = 2.50
                    Else
                        Assign toppingPrice = 0.00
                    End
                End
            End
        End
        Assign subtotal = subtotal + pizzaPrice + toppingPrice
    End
    Return Real subtotal
```

```
Function checkBlank (String text)
    Declare Boolean checkBlank

    If text==""
        Assign checkBlank = True
    Else
        Assign checkBlank = False
    End
    Return Boolean checkBlank
End
```

```
Function getDelivery
    Declare String choice
    Declare Real deliveryFee

    Do
        Output "Do you want delivery for £2.50? (Y/N):"
        Input choice
    Loop choice != "Y" and choice != "y" and choice != "N" and choice != "n"
    If choice == "Y" or choice == "y"
        Assign deliveryFee = 2.50
    Else
        Assign deliveryFee = 0.00
    End
    Return Real deliveryFee
End
```

```
Function getDetails (String Array info)
    Assign info[0] = ""
    Assign info[1] = ""
    Assign info[2] = ""
    While checkBlank(info[0])== True
        Output "Enter Name:"
        Input info[0]
    End
    While checkBlank(info[1])== True
        Output "Enter Address:"
        Input info[1]
    End
    While checkBlank(info[2])== True
        Output "Enter Phone Number:"
        Input info[2]
    End
End
```

```
Function getNumPizzas
    Declare Integer num

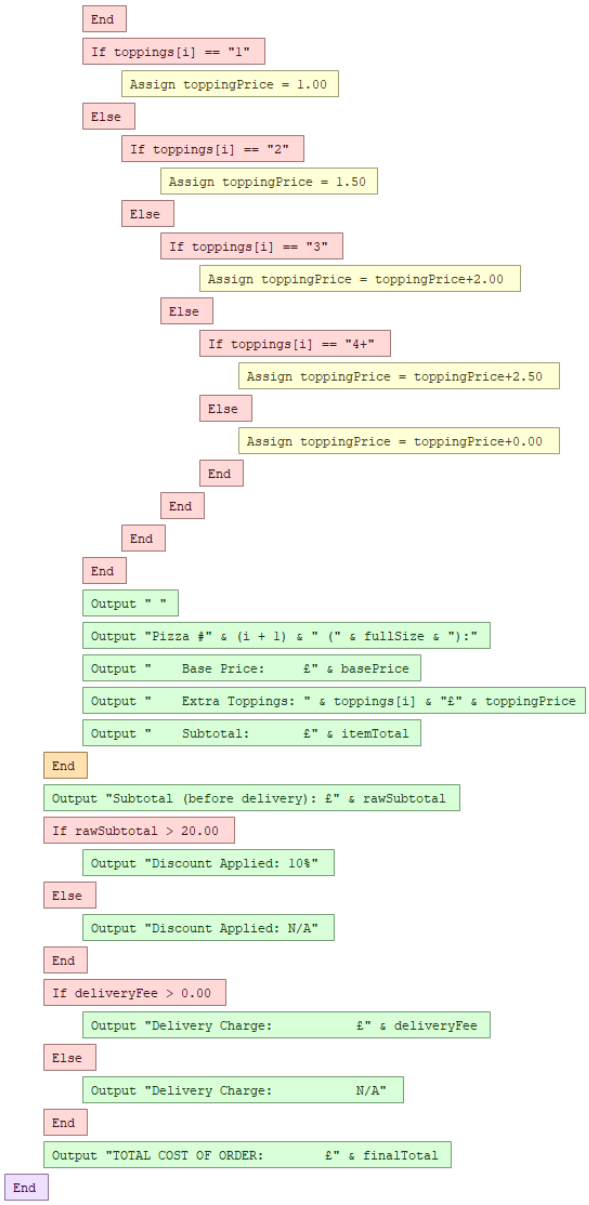
    Assign num = 0
    While num < 1 or num > 6
        Output "How many pizzas (1-6)?"
        Input num
    End
    Return Integer num
End
```

```
Function getPizzaChoices (String Array sizes, String Array toppings, Integer num)
    Declare Integer i
    Declare String wantsToppings

    For i = 0 to num-1
        Output "Pizza" & (i+1) & "Size (S/M/L):"
        Input sizes[i]
        Output "Extra toppings? (Y/N):"
        Input wantsToppings
        If wantsToppings == "Y" or wantsToppings == "y"
            Output "How many? (1, 2, 3, 4+):"
            Input toppings[i]
        Else
            Assign toppings[i] = "0"
        End
    End
End
```

```
Function printBill (String Array customerDetails, String Array sizes, String Array toppings, Integer num, Real rawSubtotal, Real discountedSubtotal, Real deliveryFee, Real finalTotal)
    Declare Integer i
    Declare String fullSize
    Declare Real basePrice
    Declare Real toppingPrice
    Declare Integer itemTotal

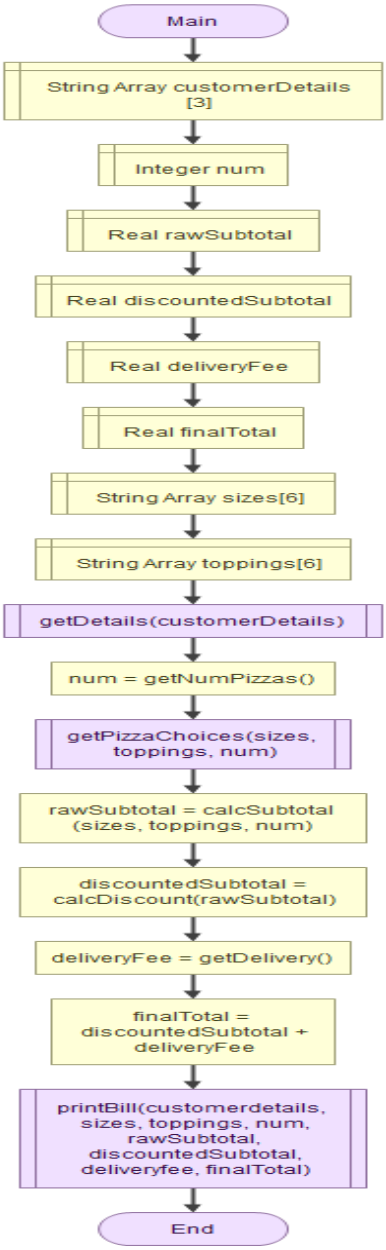
    Assign itemTotal = 0
    Assign toppingprice = 0.00
    Output "          PENNINE PIZZA WORKS          "
    Output "Name:    " & customerDetails[0]
    Output "Address:    " & customerDetails[1]
    Output "Phone Number:    " & customerDetails[2]
    Output "ORDER LIST:"
    For i = 0 to num-1
        If sizes[i] == "S" or sizes[i] == "s"
            Assign sizes[i] = "S"
            Assign fullSize = "Small"
            Assign basePrice = 3.25
        Else
            If sizes[i] == "M" or sizes[i] == "m"
                Assign sizes[i] = "M"
                Assign fullSize = "Medium"
                Assign basePrice = 5.50
            Else
                Assign sizes[i] = "L"
                Assign fullSize = "Large"
                Assign basePrice = 7.15
            End
```



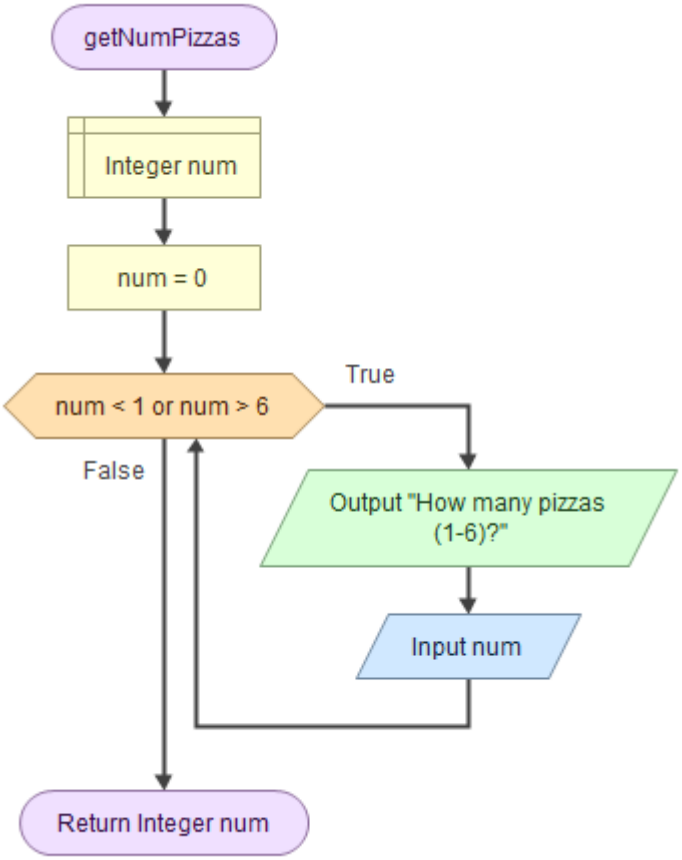
3: Build/Implementation

3.1: Program Flowcharts

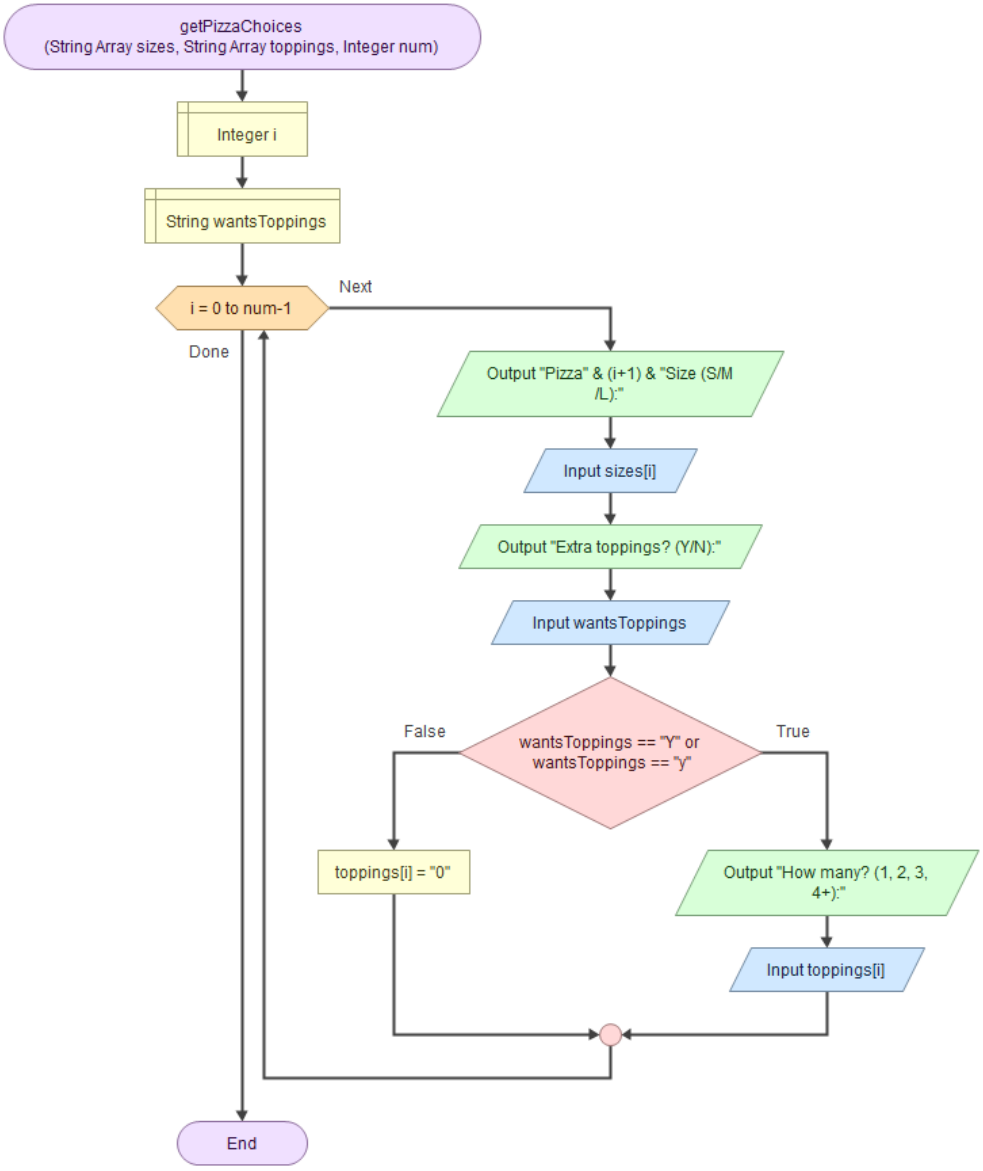
3.1.1: Main


Main
String Array customerDetails [3]
Integer num
Real rawSubtotal
Real discountedSubtotal
Real deliveryFee
Real finalTotal
String Array sizes[6]
String Array toppings[6]
getDetails(customerDetails)
num = getNumPizzas()
getPizzaChoices(sizes, toppings, num)
rawSubtotal = calcSubtotal (sizes, toppings, num)
discountedSubtotal = calcDiscount(rawSubtotal)
deliveryFee = getDelivery()
finalTotal = discountedSubtotal + deliveryFee
printBill(customerdetails, sizes, toppings, num, rawSubtotal, discountedSubtotal, deliveryfee, finalTotal)
End

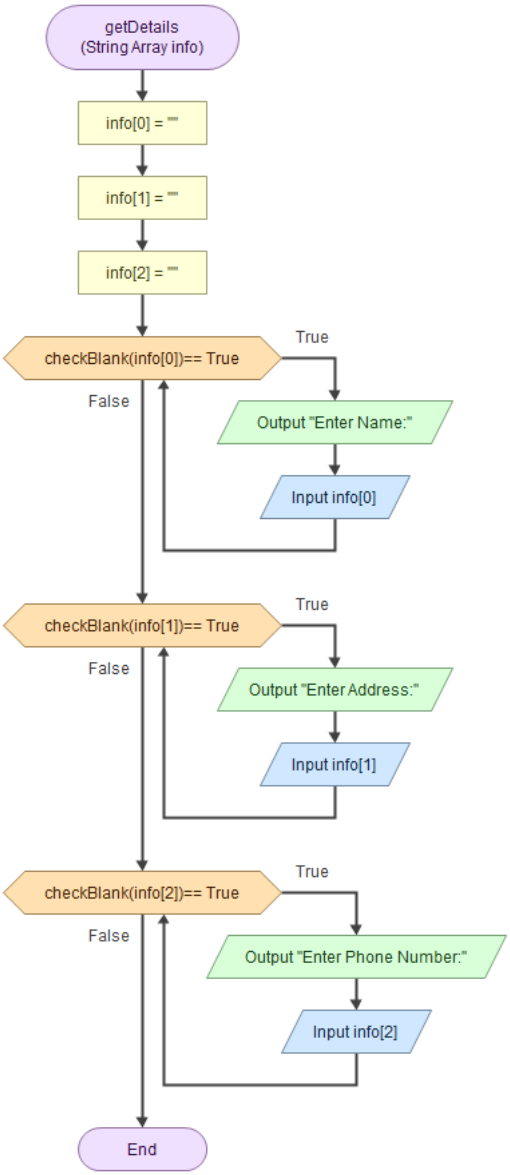
3.1.2: GetNumPizzas



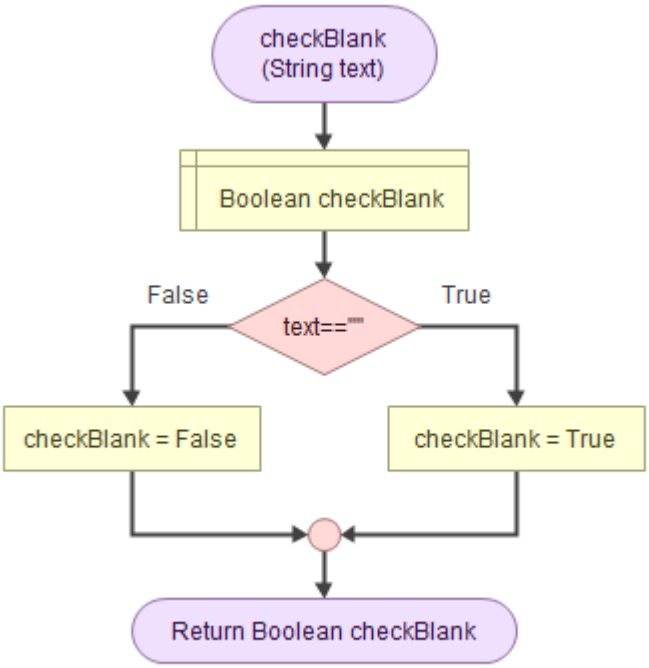
3.1.3: getPizzaChoices



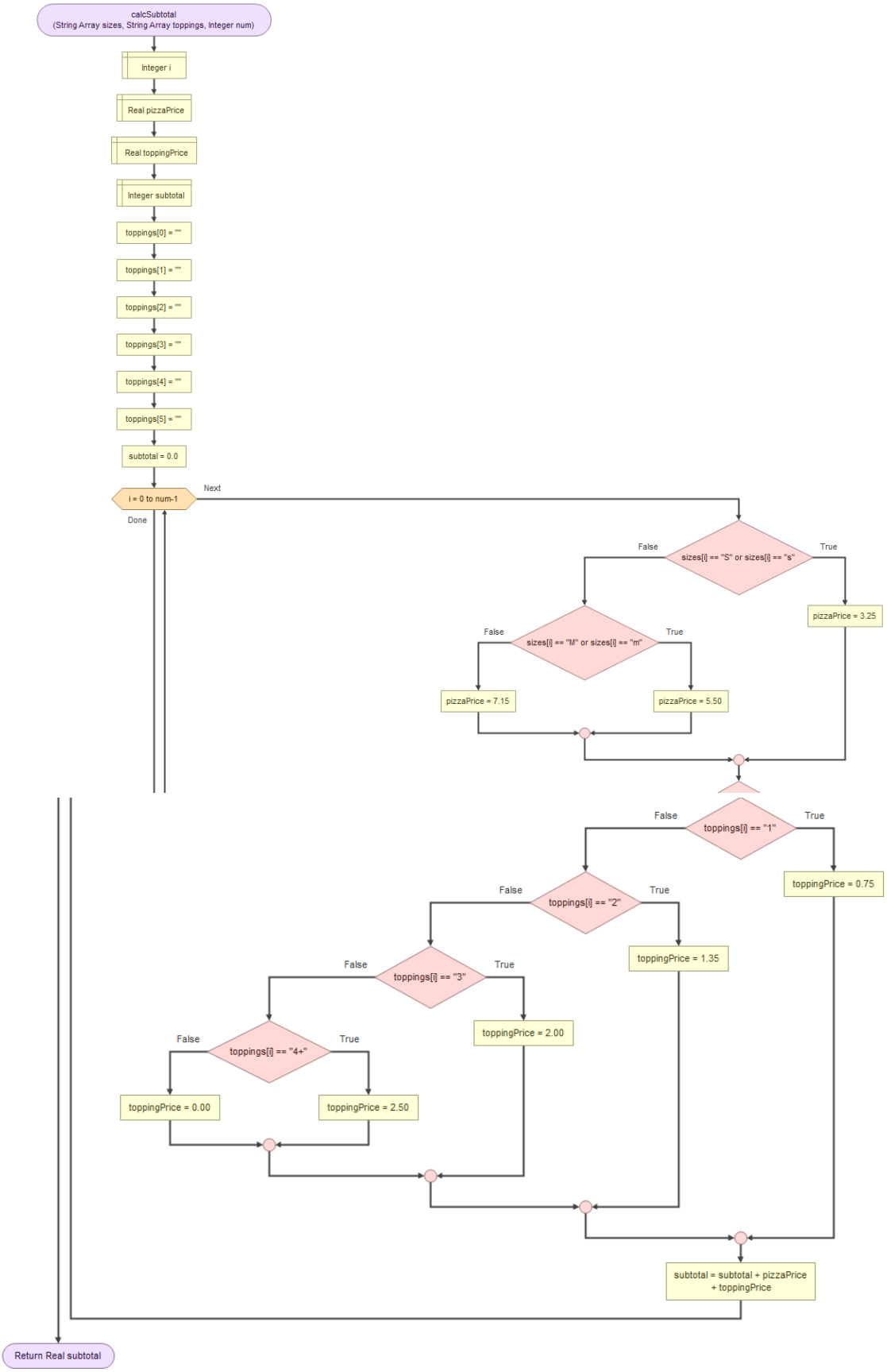
3.1.4: getDetails



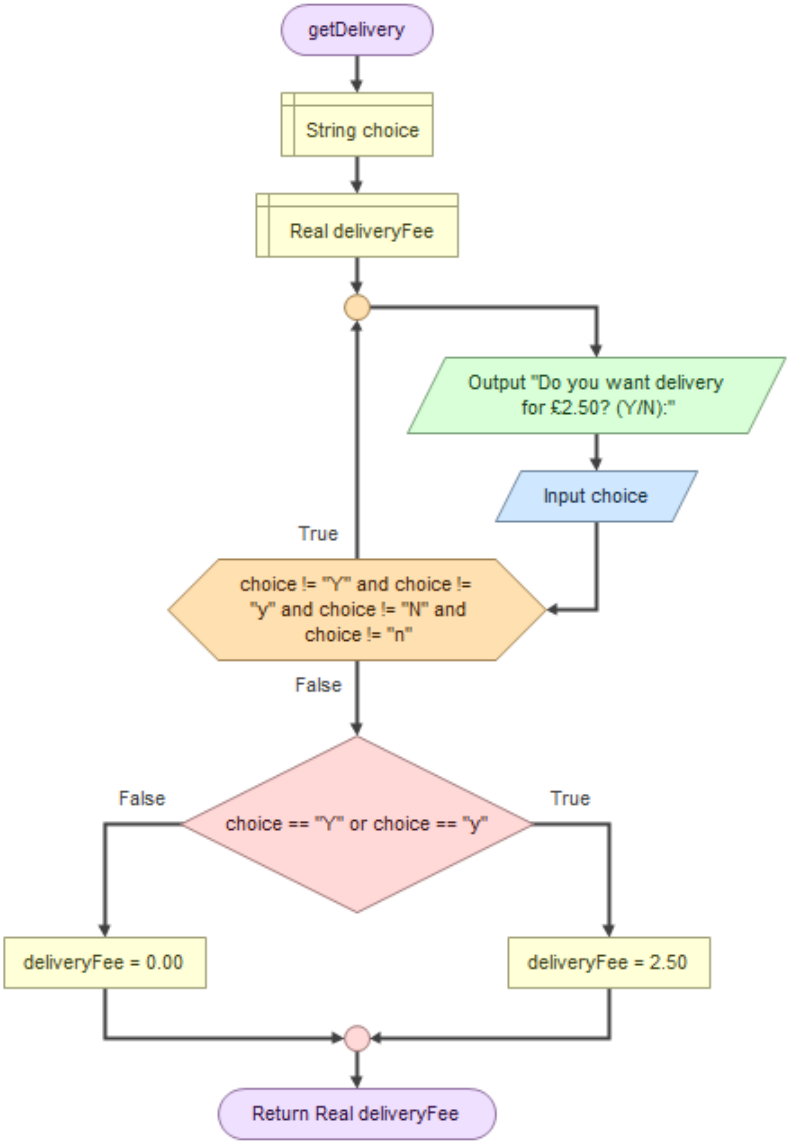
3.1.5: CheckBlank



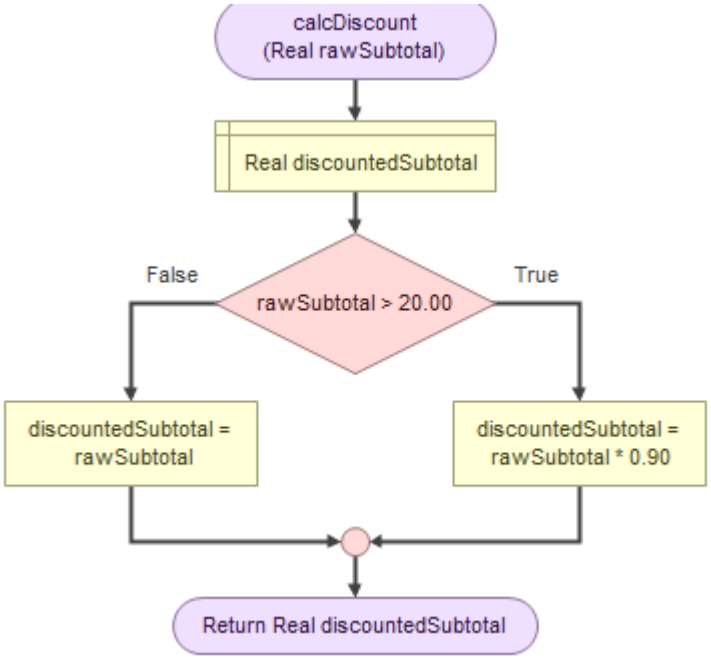
3.1.5: calcSubtotal



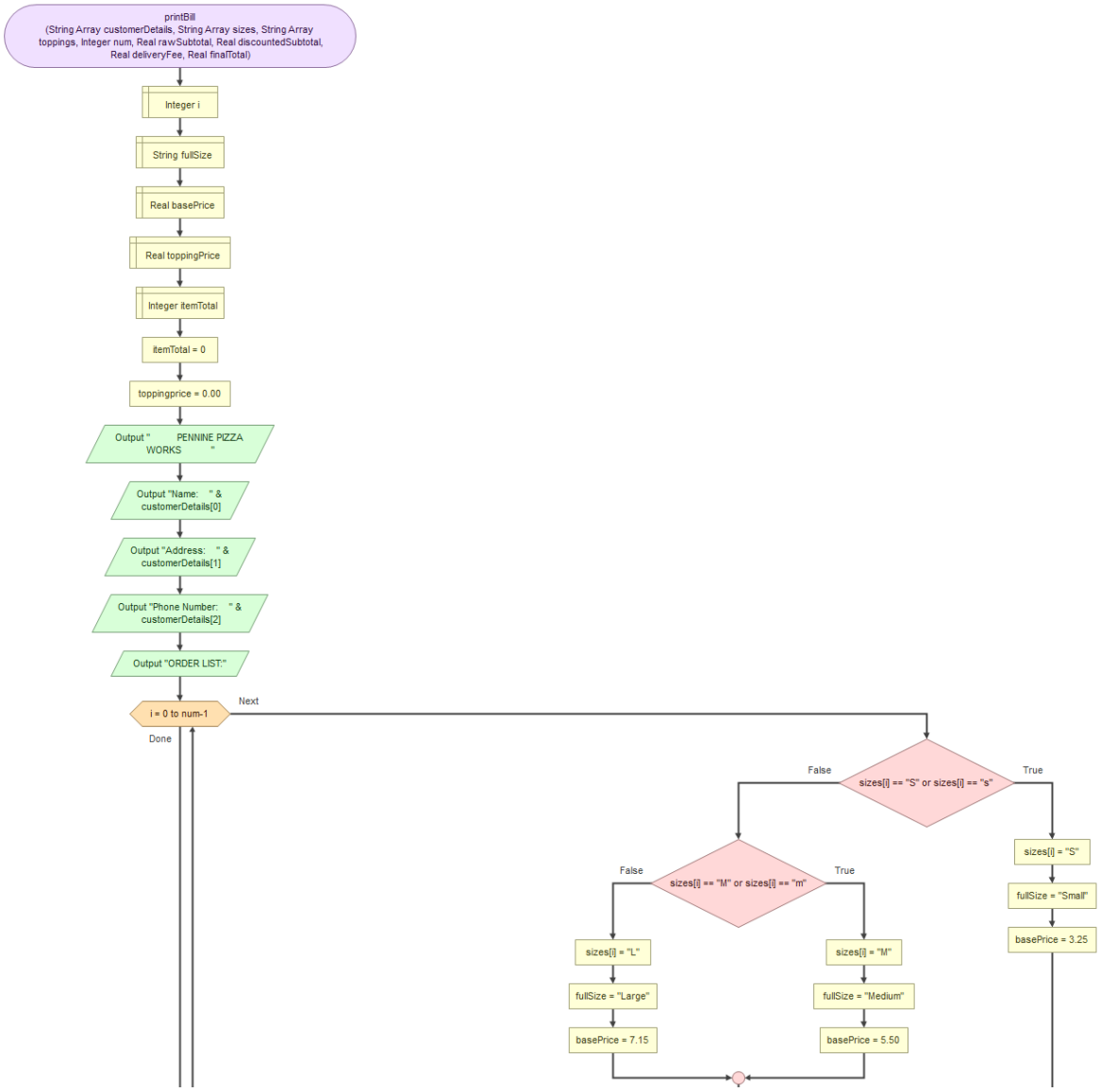
3.1.7 GetDelivery

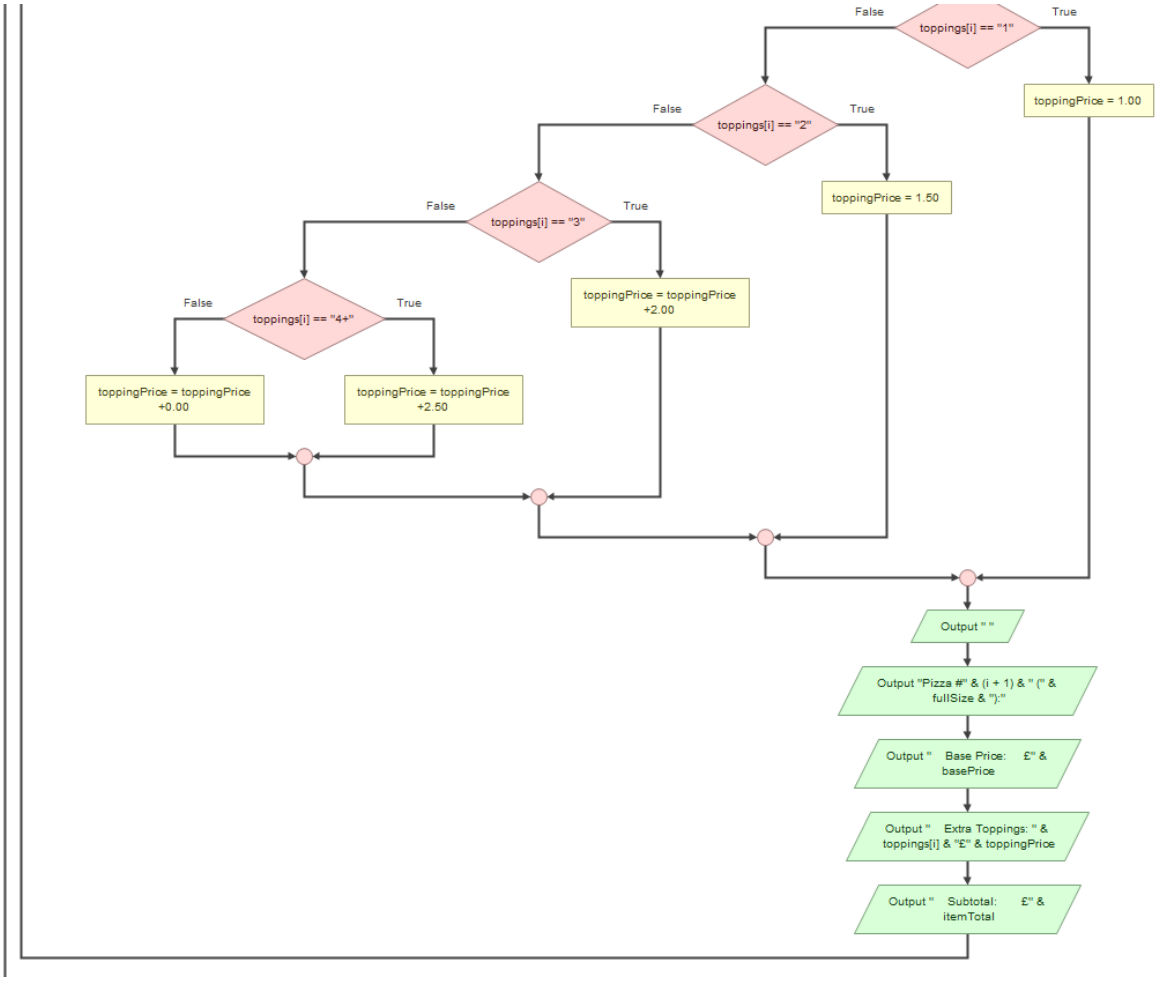

getDelivery
String choice
Real deliveryFee
Output "Do you want delivery for £2.50? (Y/N):"
Input choice
True
choice != "Y" and choice != "y" and choice != "N" and choice != "n"
False
False
choice == "Y" or choice == "y"
True
deliveryFee = 0.00
deliveryFee = 2.50
Return Real deliveryFee

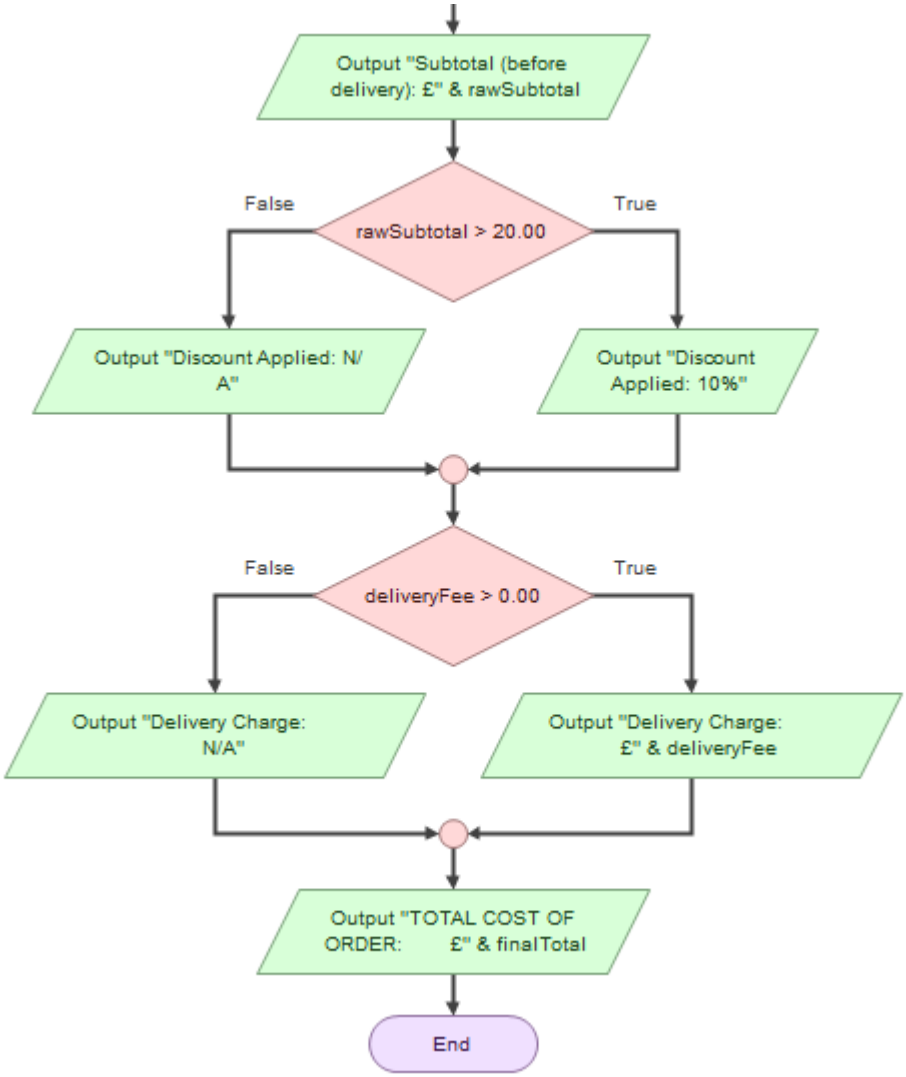
3.1.8 calcDiscount



3.1.9 printBill


printBill
(String Array customerDetails, String Array sizes, String Array toppings, Integer num, Real rawSubtotal, Real discountedSubtotal, Real deliveryFee, Real finalTotal)
Integer i
String fullSize
Real basePrice
Real toppingPrice
Integer itemTotal
itemTotal = 0
toppingprice = 0.00
Output " PENNINE PIZZA WORKS "
Output "Name: " & customerDetails[0]
Output "Address: " & customerDetails[1]
Output "Phone Number: " & customerDetails[2]
Output "ORDER LIST:"
i = 0 to num-1
Next
Done
False
sizes[i] == "S" or sizes[i] == "s"
True
sizes[i] = "S"
fullSize = "Small"
basePrice = 3.25
False
sizes[i] == "M" or sizes[i] == "m"
True
sizes[i] = "L"
fullSize = "Large"
basePrice = 7.15
sizes[i] = "M"
fullSize = "Medium"
basePrice = 5.50

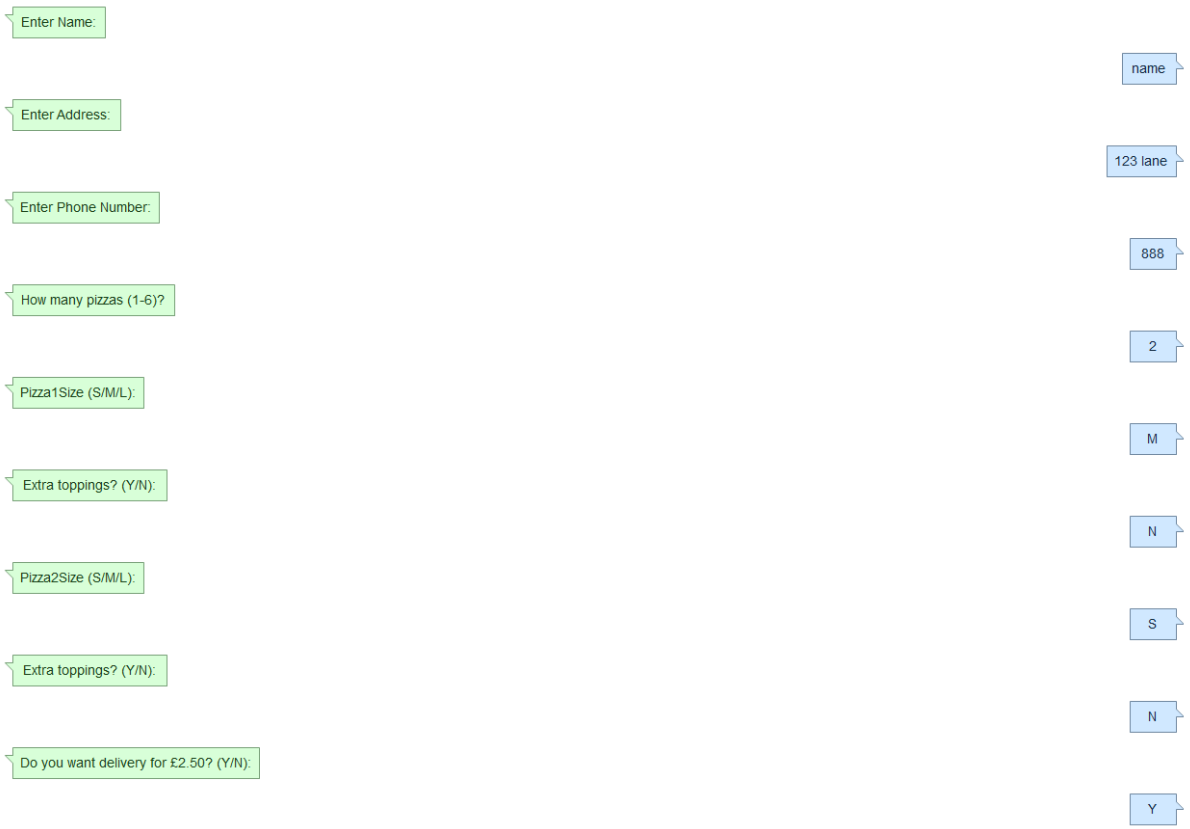

False
toppings[i] == "1"
True
toppingPrice = 1.00
False
toppings[i] == "2"
True
toppingPrice = 1.50
False
toppings[i] == "3"
True
toppingPrice = toppingPrice +2.00
False
toppings[i] == "4+"
True
toppingPrice = toppingPrice +0.00
toppingPrice = toppingPrice +2.50
Output " "
Output "Pizza #" & (i + 1) & " (" & fullSize & "):"
Output " Base Price: £" & basePrice
Output " Extra Toppings: " & toppings[i] & "£" & toppingPrice
Output " Subtotal: £" & itemTotal


Output "Subtotal (before delivery): £" & rawSubtotal
False
rawSubtotal > 20.00
True
Output "Discount Applied: N/A"
Output "Discount Applied: 10%"
False
deliveryFee > 0.00
True
Output "Delivery Charge: N/A"
Output "Delivery Charge: £" & deliveryFee
Output "TOTAL COST OF ORDER: £" & finalTotal
End

3.2: Sample Run

Screenshot of a run session showing that the code works





4: Testing

4.1: Test plan and results

Test #	Type	Purpose and test data	Expected result	Actual result	Outcome
Test 1	Normal	Test a typical valid order: 3 medium pizzas, 1 topping each, no delivery.	Each pizza = £5.50 + £0.75 topping. Subtotal = £18.75. No discount because subtotal is not over £20. No delivery charge. Final total = £18.75.	Program displays £18.75 with the correct itemised costs.	

Test 2	Extreme / Boundary	Test the minimum permitted order of 1 pizza using 1 small pizza with no toppings and no delivery.	Input of 1 pizza is accepted. Pizza cost = £3.25 and final total = £3.25.	Program accepts the minimum value and displays £3.25.	
Test 3	Extreme / Boundary	Test the maximum permitted order of 6 pizzas to ensure the upper limit is accepted.	Input of 6 pizzas is accepted, and all six pizzas are processed when calculating the order.	Program accepts all 6 pizzas and calculates the order correctly.	
Test 4	Abnormal	Enter 0 pizzas, which is below the minimum permitted value of 1.	Program should reject the value and prevent the order continuing until a valid number between 1–6 is entered.	Program rejects 0 and requests another value.	
Test 5	Abnormal	Enter 7 pizzas, which exceeds the maximum permitted value of 6.	Program should reject 7 and require a valid number between 1–6.	Program rejects 7 and requests another value.	
Test 6	Boundary	Test the discount threshold with a subtotal of	No 10% discount should be applied because the requirement	Program leaves the subtotal at £20.00 with no discount.	

		exactly £20.00.	states the subtotal must be over £20.		
Test 7	Boundary	Test an order where the subtotal is just over £20.	Program should apply a 10% discount to the subtotal, proving the discount activates only once £20 is exceeded.	Program applies the 10% discount correctly.	
Test 8	Normal	Test a valid order with delivery = Yes.	Program calculates the order first, applies any qualifying discount, then adds the £2.50 delivery charge.	£2.50 is added after the other order calculations.	
Test 9	Normal	Test a valid order with delivery = No.	No delivery charge should be added to the final order.	No £2.50 charge is added.	
Test 10	Extreme / Boundary	Test 4 extra toppings, where the highest topping-price rule begins.	Program should charge £2.50 for the extra toppings because the rule is 4 or more = £2.50.	Program applies the £2.50 topping charge correctly.	